

LLD-10: Collections

LLD-10: Python Advanced-2 — Collections

Python's built-in list and dict are great. But for specific problems, the collections module has purpose-built tools that are faster, cleaner, and less buggy.

Recap — Typing, Generics & AI Agents

Question 1: `find_user(999)` returns `None`. You call `user["name"]` without checking. When does the bug surface?

Answer

At runtime in production. Python doesn't enforce types — it happily runs `user["name"]` and crashes with `TypeError`. `mypy` would catch it *if you run it*, but `mypy` is a separate tool, not part of Python's execution. That's why you need `mypy` in CI/CD.

Question 2: `def first(items: list[T]) -> T` — you call `first(["apple", "banana"])`. What type does the IDE show?

Answer

str — TypeVar `T` is inferred as `str` from `list[str]`. The return type `T` becomes `str`. The IDE offers `.upper()`, `.split()` etc. The type flows through the TypeVar.

Question 3: You want your AI coding agent (Claude, Cursor, Copilot) to always use type hints. Where do you put these instructions?

Answer

In a **CLAUDE.md** (for Claude Code) or **.cursorrules** (for Cursor) file in the project root. The agent reads it automatically at the start of every session — like a persistent system prompt for your project. Rules like “always use type hints”, “use dataclasses not plain dicts” apply to ALL code the agent writes.

Question 4: Your team has a pre-commit hook running `mypy`. A developer writes code without type hints. What happens at `git commit`?

Answer

The commit is REJECTED. mypy returns a non-zero exit code (errors found), git refuses to create the commit. The developer must fix the type errors first. Layer 1: pre-commit hooks. Layer 2: CI/CD on the PR. Type-broken code never reaches main.

Recap — Protocol (From Last Class)

Protocol is structural typing — “if it has the right methods, it’s valid.” No inheritance needed.

```
from typing import Protocol

class Drawable(Protocol):
    def draw(self) -> str: ...

# These DON'T inherit from Drawable – they just have draw()
class Circle:
    def draw(self) -> str: return "○"

class Square:
    def draw(self) -> str: return "□"

# Accepts ANYTHING with draw() -> str
def render_all(shapes: list[Drawable]) -> None:
    for s in shapes:
        print(s.draw())

render_all([Circle(), Square()]) # Works! No inheritance needed.
```

	ABC	Protocol
Requires inheritance?	Yes	No
Works with 3rd-party?	No (must wrap)	Yes
Runtime check?	isinstance()	@runtime_checkable
Analogy	Java interface	Go interface
Use when	You own the hierarchy	You need a contract for any class

defaultdict — No More KeyError

The Problem

```
# Grouping words by first letter with plain dict:
grouped = {}
for word in words:
```

```

first = word[0]
if first not in grouped:    # boilerplate every time!
    grouped[first] = []
grouped[first].append(word)

```

The Fix

```

from collections import defaultdict

grouped = defaultdict(list)    # missing key → empty list
                                # automatically
for word in words:
    grouped[word[0]].append(word)    # no if-check!

```

Factory Functions

Factory	Missing key returns	Use case
<code>defaultdict(list)</code>	<code>[]</code>	Grouping items by key
<code>defaultdict(int)</code>	<code>0</code>	Counting occurrences
<code>defaultdict(set)</code>	<code>set()</code>	Collecting unique values
<code>defaultdict(lambda: "N/A")</code>	<code>"N/A"</code>	Custom default value

Gotcha: Accessing a missing key **creates** it. `d["x"]` adds "x" to the dict even if you're just reading. Use `d.get("x")` or `"x" in d` to check without creating.

Counter — Count Everything

```

from collections import Counter

words = "the cat sat on the mat the cat".split()
counts = Counter(words)
# Counter({'the': 3, 'cat': 2, 'sat': 1, 'on': 1, 'mat': 1})

counts['the']    # 3
counts['dog']    # 0 (not KeyError!)
counts.most_common(2)  # [('the', 3), ('cat', 2)]

```

Counter Arithmetic

```

a = Counter(apples=3, bananas=2)
b = Counter(apples=1, bananas=4)

a + b    # Counter(apples=4, bananas=6) — combine

```

```
a - b # Counter(apples=2)           - subtract (drops ≤0)
a & b # Counter(apples=1, bananas=2) - minimum
a | b # Counter(apples=3, bananas=4) - maximum
```

Real-world: word frequency, inventory tracking, vote counting, log analysis, feature usage metrics.

deque — Fast From Both Ends

The Problem

```
# list.insert(0, x) is O(n) - shifts every element!
# For 100,000 items: list takes ~2s, deque takes ~0.002s
# deque is 1000x faster for prepend operations
```

The Fix

```
from collections import deque

dq = deque([1, 2, 3])
dq.append(4)           # right: O(1)
dq.appendleft(0)        # left:  O(1)
dq.pop()               # right: O(1)
dq.popleft()           # left:  O(1)
```

maxlen — Sliding Window

```
recent = deque(maxlen=3)
for item in ["a", "b", "c", "d", "e"]:
    recent.append(item)
# Only last 3 kept: deque(['c', 'd', 'e'])
```

Performance Comparison

Operation	list	deque
Append right	O(1)	O(1)
Append left	O(n)	O(1)
Pop right	O(1)	O(1)
Pop left	O(n)	O(1)
Random access [i]	O(1)	O(n)

Use deque for: queues (FIFO), sliding windows, BFS, recent-N items. **Don't use for:** random access by index.

namedtuple — Readable Tuples

The Problem

```
point = (28.6139, 77.2090)
point[0] # What is this? x? latitude? id? WHO KNOWS!
```

The Fix

```
from typing import NamedTuple

class Point(NamedTuple):
    x: float
    y: float

p = Point(28.6139, 77.2090)
p.x      # 28.6139 – readable!
p[0]     # still works as tuple
p.x = 10 # AttributeError – immutable!
```

Useful Methods

```
user = User("Alice", 25, "alice@example.com")
user._replace(age=26) # new User with age changed (original
                      # unchanged)
user._asdict()        # {'name': 'Alice', 'age': 25, 'email':
                      # '...'}
name, age, email = user # unpacking works
```

namedtuple vs dataclass

	namedtuple	dataclass
Mutable?	No (immutable)	Yes (by default)
Is a tuple?	Yes (unpackable, indexable)	No
Methods?	Only <code>_replace</code> , <code>_asdict</code>	Full custom methods
Use for	Small immutable records	Larger structures, need methods

OrderedDict — Order-Sensitive Equality

```
from collections import OrderedDict

# Regular dict: order doesn't matter for ==
{"a": 1, "b": 2} == {"b": 2, "a": 1} # True
```

```
# OrderedDict: order IS part of equality
OrderedDict([("a", 1), ("b", 2)]) == OrderedDict([("b", 2), ("a",
1)]) # False!
```

Key methods: `move_to_end(key)`, `popitem(last=False)`. Great for **LRU caches**.

ChainMap — Layered Config

```
from collections import ChainMap

defaults = {"theme": "dark", "font_size": 14}
env = {"font_size": 16, "debug": True}
user = {"theme": "light"}

config = ChainMap(user, env, defaults)
config["theme"]      # "light" (user wins)
config["font_size"]  # 16 (env wins)
config["debug"]      # True (env)
```

Lookup order: first dict → second → third. No data copied. Real-world: Django settings layering, CLI arg overrides, template variable scopes.

frozenset — Immutable Set

```
# Sets can't be dict keys:
{"python", "backend": "team_a"} # TypeError!

# frozensets can:
tags = frozenset({"python", "backend"})
{tags: "team_a"} # Works! frozenset is hashable.

# All set operations work (return new frozensets):
tags_a = frozenset({"python", "backend"})
tags_b = frozenset({"python", "ml"})
tags_a | tags_b # union
tags_a & tags_b # intersection
```

Use for: dict keys when key is a set, cache keys, graph edges, immutable permissions.

Custom Collections — UserDict, UserList, UserString

Why not subclass dict directly?

```
class BrokenUpperDict(dict):
    def __setitem__(self, key, value):
        super().__setitem__(key.upper(), value)

d = BrokenUpperDict()
d["hello"] = 1          # Uses __setitem__ → "HELLO"
d.update({"world": 2})  # BYPASSES __setitem__! → "world"
                        (lowercase!)
# BUG: update() doesn't go through __setitem__ in dict!
```

The Fix: UserDict

```
from collections import UserDict

class UpperDict(UserDict):
    def __setitem__(self, key, value):
        super().__setitem__(key.upper(), value)

d = UpperDict()
d["hello"] = 1
d.update({"world": 2})  # Goes through __setitem__!
# {'HELLO': 1, 'WORLD': 2} — correct!
```

UserList — validated list (e.g., only positive numbers):

```
class PositiveList(UserList):
    def append(self, item):
        if item <= 0:
            raise ValueError(f"Only positive numbers! Got {item}")
        super().append(item)
```

UserString — custom string behavior (e.g., case-insensitive comparison):

```
class CaselessString(UserString):
    def __eq__(self, other):
        return self.data.lower() == str(other).lower()
```

Rule: NEVER subclass dict, list, or str directly. Internal C methods bypass your overrides. Use UserDict, UserList, UserString instead.

Thread Safety in Collections

What's Safe Under the GIL?

Safe (single bytecode)	NOT Safe (compound operation)
<code>list.append(x)</code>	<code>counter += 1</code> (LOAD + ADD + STORE)
<code>list.pop()</code>	<code>if key not in d: d[key] = val</code> (check-then-act)
<code>dict[k] = v</code>	<code>list[i] = list[i] + 1</code>
<code>deque.append(x)</code>	Any read-modify-write
<code>deque.popleft()</code>	
<code>set.add(x)</code>	

For Thread-Safe Queues: `queue.Queue`

```
import queue
```

```
q = queue.Queue()
q.put("item")           # thread-safe, blocks if full
item = q.get()          # thread-safe, blocks if empty
q.task_done()
```

Don't rely on the GIL for thread safety. It's a CPython implementation detail that may be removed (PEP 703 — free-threaded Python). Use `threading.Lock` or `queue.Queue` for correctness.

Which Collection for Which Job?

I need to...	Use
Group items by key	<code>defaultdict(list)</code>
Count occurrences	<code>Counter</code>
Fast prepend / pop from front	<code>deque</code>
Sliding window / last N items	<code>deque(maxlen=N)</code>
BFS queue	<code>deque(popleft)</code>
Immutable record with named fields	<code>NamedTuple</code>
Layered config (defaults + overrides)	<code>ChainMap</code>
Use a set as a dict key	<code>frozenset</code>
Order-sensitive equality	<code>OrderedDict</code>
LRU cache	<code>OrderedDict</code>
Custom dict behavior	<code>UserDict</code> subclass
Validated list	<code>UserList</code> subclass
Thread-safe queue	<code>queue.Queue</code>

Mutability Pairs

Mutable	Immutable
list	tuple
set	frozenset
dict	MappingProxyType
bytearray	bytes

Code Files

File	What It Demonstrates
01_protocol_recap.py	Protocol vs ABC — structural typing
02_defaultdict.py	defaultdict(list/int/set), gotchas
03_counter.py	Counter, most_common, arithmetic
04_deque.py	deque benchmark, maxlen, rotate
05_namedtuple.py	NamedTuple, _replace, _asdict, vs dataclass
06_chainmap.py	ChainMap layered config, new_child
07_frozenset.py	frozenset as dict key, set operations
08_orderdict.py	OrderedDict equality, LRU cache
09_custom_collections.py	UserDict, UserList, UserString — why dict subclassing breaks
10_thread_safety.py	Safe vs unsafe operations, queue.Queue
11_which_collection.py	Decision guide — which collection for which job

Key Takeaways

1. **defaultdict eliminates KeyError boilerplate** — factory functions (list, int, set) auto-create missing keys
2. **Counter counts anything** — most_common(), arithmetic, missing keys return 0
3. **deque is 1000x faster than list for prepend** — use for queues, BFS, sliding windows
4. **namedtuple = tuple + readability** — immutable, unpackable, typed
5. **Never subclass dict/list/str directly** — use UserDict/UserList/UserString
6. **GIL makes single operations atomic but don't rely on it** — use queue.Queue for thread-safe communication

Next class: Python Advanced-3 — Lambda Functions and FP